

CI-CD Pipeline and Process Management

- **CI-CD Pipeline and Process Management**
 - **Process Management**
 - **Key Features**
 - **Installing pm2 with some useful terminal command**
 - **What is CI/CD ??**
 - **Continuous Integration**
 - **Continuous Deployment**
 - **CD in Github**
 - **Architecture diagram**
 - **Monorepo we're dealing with in this notes**
 - **How to create CI/CD Pipeline**
 - **Create the CI pipeline**
 - **Building a pipeline for the repo**
 - **Lets add a deploy step**
 - **Pulling the docker image**
 - **Assignments**

Process Management

generally called as **PM2**

Keeps a process running unless stopped explicitly, very common feature present in machine such as EC2.

Key Features

- **Keeps apps running continuously**
- **Automatically restarts crashed apps**
- **Provides monitoring and logging**
- **Supports running multiple app instances for better performance**

The above is what the process management does and even the above is what it actually means

Installing pm2 with some useful terminal command

For installing **pm2** on your VM, run the below command

```
npm i -g pm2
// but as mostly your VM has OS as ubuntu selected hence
sudo npm i -g pm2 // this will work for linux based os as in linux most of the
thing, you have to give permissions as SUPERUSER and hence "SuperuserDO" command
is tagged at start to forcefully install anything
```

The best thing which will happen after this is that even though now if you will kill the process, **with the help of pm2, the process will still be running and its instance will be stored.**

You can check the status of your process which has pm2 installed by running the below command

```
pm2 status

// OR

pm2 ls // this is better as it will give all the pm2 installed process list with
their active status
```

You will still see the **Online** status of the process you killed by using the **kill** command.

Even though the server stopped for some hardware failure and pm2 still not able to restart the server automatically then you can just do

```
pm2 start index.js // so instead of using node index.js for running the project
you will run this
```

and to **stop the pm2** just run the command

```
pm2 stop
```

Basically no one can stop the malfunction inside the server which can lead to server stopping but we can atleast stop the MANUAL restarting of the server once it again comes online and this is what the pm2 does

Pm2 ensures that whenever the server will get restored, it will automatically restarts the process present on the server so that you dont have to manually start it and also you dont have to check repeatedly that whether server restored or not so that you can restart the process, PM2 will handle that



to get all the commands of pm2 just run the command **pm2 examples**

```
lsof -i :3000 // will give you the process running on some port (here on 3000) //
it has PROCESS ID

// if you do
lsof // just this then you will get a list of all the process with their PROCESS
ID that are running on what port

// and if you want to kill the process running on particular port just use
kill process_id_of_process_you_are_trying_to_kill
```

Also if you want to see the output of the server (or basically **response of the server which is running**)

use the below command

```
curl your_link
```

curl command basically acts **as the POSTMAN but for the terminal** so instead of using postman and if you are inside the terminal and want to see the output or response of the server then just use this **curl** command.

What is CI/CD ??

Continuous Integration

Continuous Integration (CI) is a development practice where **developers frequently integrate their code changes into a shared repository**, preferably several times a day. Each integration is automatically verified by

1. Building the project and
2. **Running automated tests.**

basically continuously running and integrating code into the repo on some version control systems like github

This process allows teams to detect problems early, improve software quality, and reduce the time it takes to validate and release new software updates.

Or in short, before people push anything the maintainer can **run a workflow and can see if the code they are trying to merge in your repo is correctly LINTED, or FORMATTED**

Continuous Deployment

As the name suggests, deploying your code **continuously and automatically** to various environments (dev/stage/prod).

CD in Github

We'll be deploying a next.js app to EC2 server via docker

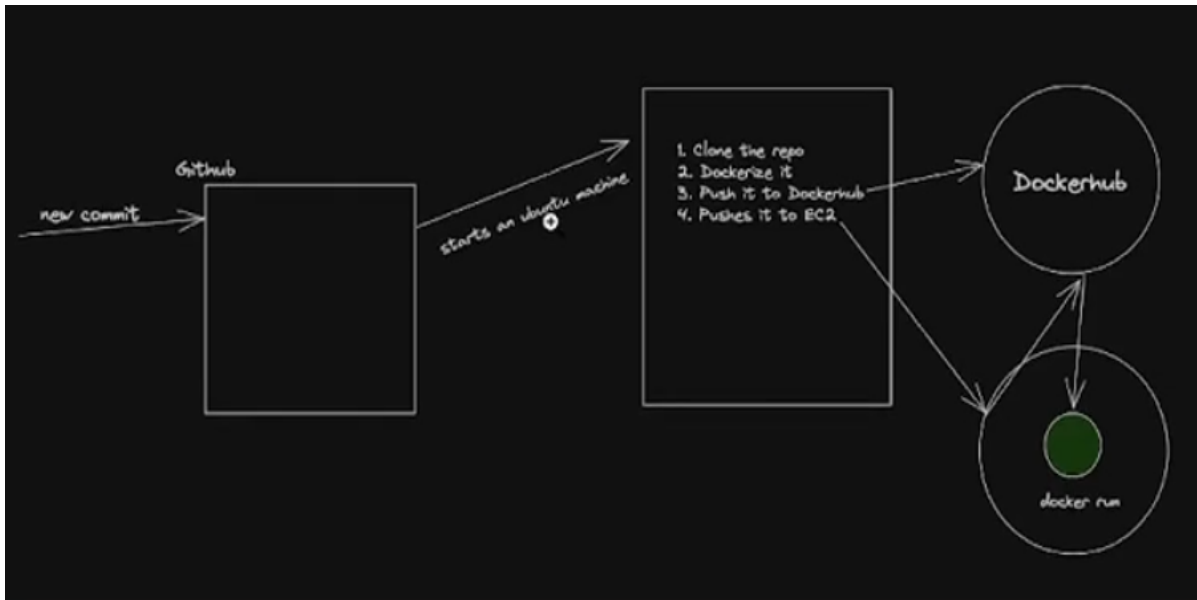


You don't really need Docker here, since it's deploying on a simple EC2 server. If you deploy to

1. GCP App runner
2. ECS
3. Kubernetes

then it makes more sense to make it **dockerised**

Architecture diagram



basically the above picture says that whenever a new commit will come to the repo, github will make sure to **run some steps (mentioned as numbers) also known as WORKFLOW**, these steps are run on a machine (you actually bring a small machine for some time to run these steps so that code can be pushed to their correct place)

Monorepo we're dealing with in this notes

The projects we are deploying with CI/CD pipeline is [CI-CD Example](#)

This monorepo has 3 apps inside :-

1. bank-webhook
2. merchant-app
3. user-app

We'll be deploying one (user-app) of these three to the same EC2 instance.[if you learn one then you just have to **copy paste the steps to deploy all of them in the same EC2 instance**]

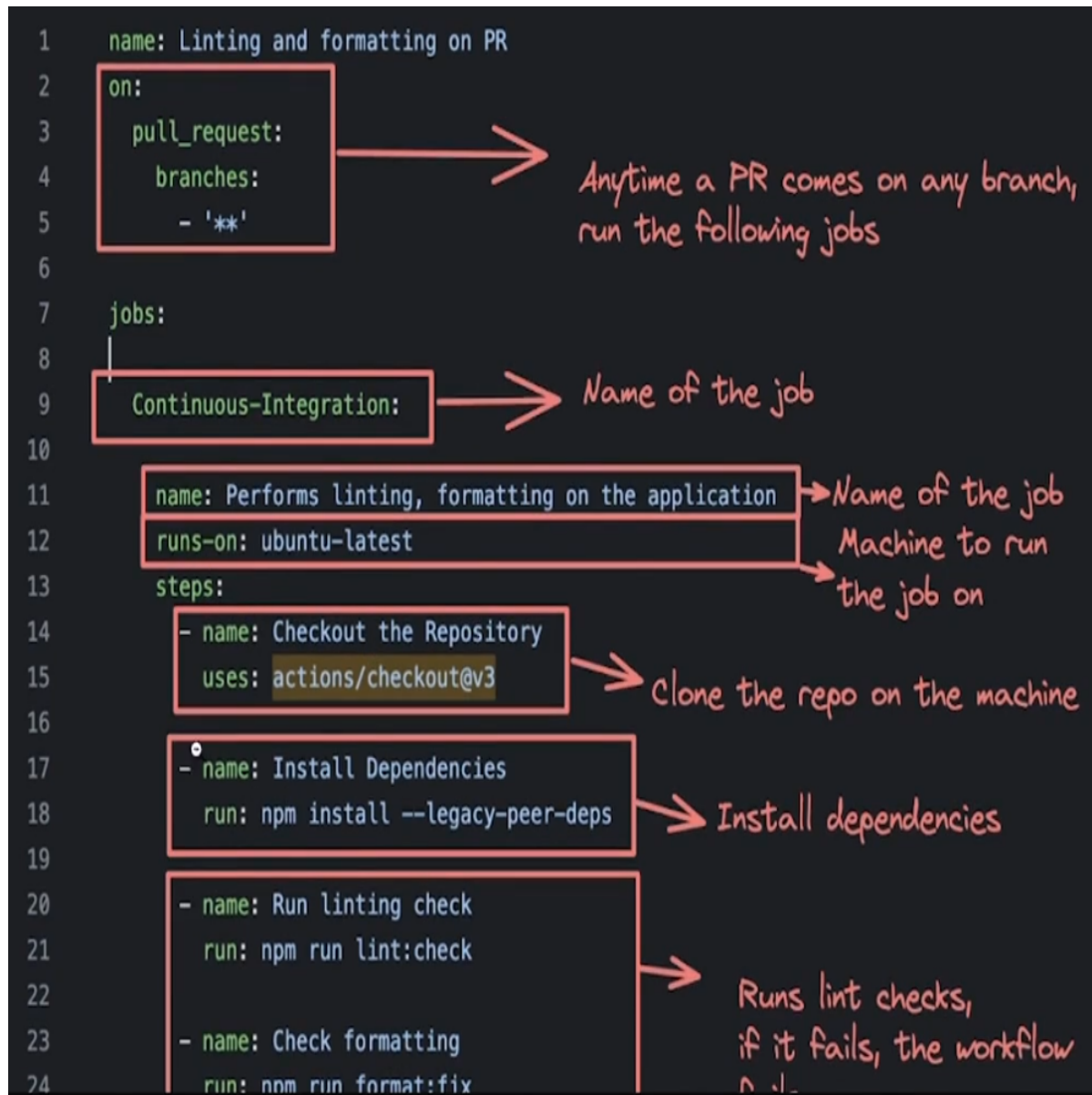
How to create CI/CD Pipeline

For github, **you can add all your pipelines to `.github/workflows`**[the place where all the steps are defined]

Lets first focus on integrating the code i.e. -> CI (anytime anyone is doing pull request, i should run a workflow that makes sure the code is **building correctly**)

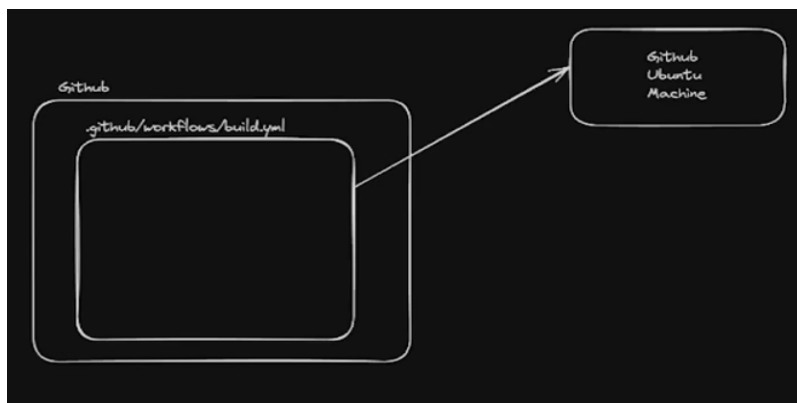
For ex -> [Workflow example](#)

Explaining the code for a particular pipeline ->



Create the CI pipeline

Make sure that whenever someone tries to create PR, we build the project and make sure that it builds as expected.



Building a pipeline for the repo

Anytime a user creates a PR, we need to run `npm run build` and only if it succeeds should the workflow succeed.

- **Fork** the main repo -> [Repo](#)
- **Add `.github/workflows/build.yml` in the root folder**
- **Create the workflow** :- (inside the `build.yml` file)

```
name: Build on PR

on: // When should this run ?? Answer is below
  pull_request: // whenever a pull request is made on
    branches:
      - master // master branch
      - dev // you can add more branches here

jobs:
  build: // As we have to do a single job (to build)
    name : Build the project // name given to the job
    runs-on: ubuntu-latest // machine on which this will run
    steps: // All the steps which you want to be done
      - uses: actions/checkout@v3 // Clones the repository
      - name: Use Node.js
        uses: actions/setup-node@v3 // Install node.js on the machine
        with:
          node-version: '20'

      - name: Install Dependencies
        run: npm install // run npm install then

      - name: Generate Prisma client
        run: npm run db:generate // if the project is using prisma then you have
        generate its client also to make the project work // 2

      - name: Run Build
        run: npm run build // and finally run npm run build

// Also learn about actions/---something-- to know about what they do from the
internet
```

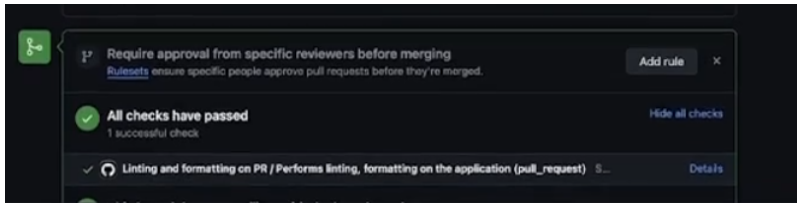
make sure to add the `db:generate` required command in the `package.json` file which is

```
"scripts":{
  "db:generate": "cd packages/db && npx prisma generate && cd ../../"
}
```

✖ **Its not important to learn or memorise the steps, you can GOOGLE what are all the steps you require and tweak them according to the need.**

No need to be ashamed of copying the things and making changes in it

- **Push this to master branch**
- Create a **new branch** with some minimal changes and create PR from it
- You should see the workflow run



Now generally you will see this file being very strict like **adding test so that if a certain pull request does not pass an appropriate number of tests then that pull request will automatically get discarded and ignore the merge request of that**

That's how you CREATE A CI PIPELINE

Lets add a deploy step

Now that **CI has been done**, lets understand **CD**

For this you must be having a VM machine. and also you can do the below things without using the docker (by installing all the required things for your project like -> **node.js** installed, **prisma** installed and so on.. on your machine).

- **Create dockerfiles for the apps you have**

Create docker/Dockerfile.user (basically all the docker related files are kept inside the **docker** named folder(you can name anything as per your wish))

```
FROM node:20.12.0-alpine3.19

WORKDIR /usr/src/app

COPY package.json package-lock.json turbo.json tsconfig.json ./

COPY apps ./apps
COPY packages ./packages

# Install dependencies
RUN npm install
# Can you add a script to the global package.json that does this?
RUN cd packages/db && npx prisma generate && cd ../..
# OR instead of the above line you can also replace it with
RUN npm run db:generate # see above if you are not able to understand how this replacement came
```

```
# Can you filter the build down to just one app?
RUN npm run build

CMD ["npm", "run", "start-user-app"]
```

- **Add start-user-app script to the root package.json** (basically you have added a script that builds this project)

```
"start-user-app": "cd ./apps/user-app && npm run start"
```

The above were the steps to **containerise your project**

 **For every single project you should have separate DOCKERISED file, you cannot make a single docker file even though it is monorepo**

Now running the below command will

```
docker build -t mynextapp -f docker/Dockerfile.user .
```

containerise our project

Now after the above command is executed, **running the project**

```
docker run -p 3000:3000 mynextapp
```

and this will run your project locally

Now the next step is to again add a workflow that runs every time a commit is added to the **master** branch.

The deployment should not happen when someone creates a pull request, deployment should happen when someone commits and that too on the **master** branch, then only deployment should occur.

In industry there is another branch named as **dev** and all of the code is first committed on this branch and then eventually deploy to a copy app and then in the interval of 2 weeks (generally), you merge all the code from **dev** branch to the **master** branch and then it released to the production. **These are called as RELEASE CYCLES.** [Every 2 weeks, you are **releasing dev** to **master**]

Now after **dockerising** you now have to **Create the CD pipeline that**

- **clones the repo**
- **Builds the docker file**
- **Pushes the docker image**

If you see the architecture then step 4 -> Pushes to the EC2 will not be done for now as it changes for different platforms

and for the above task as it is automation related task so make another file named as `deploy.yml` inside the `.github/workflows`

So creating the CI/CD pipeline for the above 3 tasks

```
name: Build and Deploy to Docker Hub

on:
  push:
    branches:
      - master // Now you have to push to master branch only to proceed further as
this is CI for deployment part

jobs:
  build-and-push: // name of the job
    runs-on: ubuntu-latest
    steps:
      - name: Check Out Repo
        uses: actions/checkout@v2

      - name: Log in to Docker Hub
        uses: docker/login-action@v1
        with:
          username: ${ secrets.DOCKER_USERNAME }}
          password: ${ secrets.DOCKER_PASSWORD }}

      - name: Build and Push Docker image
        uses: docker/build-push-action@v2
        with:
          context: .
          file: ./docker/Dockerfile.user
          push: true
          tags: 100xdevs/web-app:latest # Replace with your Docker Hub username
and repository

      // The below is optional automation step
      - name: Verify pushed image
        run: docker pull 100xdevs/web-app:latest # Replace with your Docker Hub
username and repository
```

Again the above code has been googled to very extent basically the steps on how to use docker in the CI/CD pipeline and many others, and again there is no problem in doing that

ofcourse for the above task, you have to create a repo on the dockerhub as **github dockerhub pe files and folder kis jagah pe push krega** so go to the website and create a repo

Now in the above repo we have create on the dockerhub, github will do the above 3 task and on the ubuntu machine and then push it to the dockerhub

Now if the github is pushing the files to dockerhub, then **it must have your credentials of the dockerhub**

💡 How to generate the credentials in the dockerhub for github ??

- Go to the dockerhub dashboard
- Inside account dropdown menu select **My Account**
- Click on the **Security** option then
- and then on **New Access Token** and then **github** and give the permissions to **Read, write and delete** (basically github has access to this file config type) and then click on **generate**
- and finally you will get an ACCESS TOKEN (this is your **password in other words**)

Now 💡 How to give this password to github ??

- Go to the your repo **settings** (repo which is doing the deployment)
- go to the option **Secrets and variables**, and then **actions** [as your github actions is what pushing to dockerhub]
- Now click on the **Repository Secrets**
- Give the name **DOCKERHUB_USERNAME** and secrets as value **100xdevs** (whatever your username is on the dockerhub)
- finally click on the **Add secret** button to add the username to the github
- Similar to the above we will make another **Repository secrets** and give it the name **DOCKER_PASSWORD** and secrets value as the same as that of the **Access token** which you got from the dockerhub
- and hence you have given both username and password to the github to interact it with dockerhub

-> **The reason you have not put their values directly in the workflow folder of github (although this is also automation part) is that you dont want your username and password to be made available to every person in the world**

and now if you want to access them then you can access them by using `${{secrets.DOCKER_USERNAME}}` and `${{secrets.DOCKER_PASSWORD}}` (as seen and used in the above code)

Now coming to the last step **Pushing to the EC2 machine** and setting up this

Now you have machine, you just have to install **docker** inside it and then you are good to go, you can now run your project without installing **node.js**, **prisma cli** and other requirements to run the project on the machine as **docker** will handle it

Pulling the docker image

Link to documentation -> [Pulling the docker image](#)

- **Create an EC2 server**
 - **Download its keypair file**
 - **Allow http/https traffic**
 - **Ubuntu base image**
- **Download docker on the machine**

- <https://docs.docker.com/engine/install/ubuntu/>
- `sudo docker run hello-world` // to check whether docker is installed on your machine or not if you get Hello from Docker! msg you are good to go

- **Just update the workflow to pull the latest image on the ec2 machine**

```
name: Build and Deploy to Docker Hub

on:
  push:
    branches:
      - master

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - name: Check Out Repo
        uses: actions/checkout@v2

      - name: Log in to Docker Hub
        uses: docker/login-action@v1
        with:
          username: ${ secrets.DOCKER_USERNAME }
          password: ${ secrets.DOCKER_PASSWORD }

      - name: Build and Push Docker image
        uses: docker/build-push-action@v2
        with:
          context: .
          file: ./docker/Dockerfile.user
          push: true
          tags: 100xdevs/web-app:latest // Replace with your Docker Hub username
and repository

      - name: Verify pushed image
        run: docker pull 100xdevs/web-app:latest // Replace with your Docker Hub
username and repository

      // JUST ADD THE BELOW LOGIC TO THE EXISTING ONE TO MAKE THE WORKFLOW
HANDLE PUSHING TO THE GITHUB ALSO
      - name: Deploy to EC2
        uses: appleboy/ssh-action@master
        with:
          host: ${ secrets.SSH_HOST }
          username: ${ secrets.SSH_USERNAME }
          key: ${ secrets.SSH_KEY }
        script: | // given the above info run the below commands
          sudo docker pull 100xdevs/web-app:latest
          sudo docker stop web-app || true
          sudo docker rm web-app || true
```

```
sudo docker run -d --name web-app -p 3005:3000 100xdevs/web-app:latest
// Replace with dockerhub username repo here // 2
```

Now again the same thing if you want your github to interact with your EC2 machine or basically VM, then you should give the credentials of your machine to the github and that will be **added in the secrets making tab in the github as seen above (how to add secrets)** and `${{ secrets.SSH_HOST }}`, `${{ secrets.SSH_USERNAME }}` and `${{ secrets.SSH_KEY }}` are the credentials of your VM machine only whose values is store in the **Repository secrets** in github.



What values i have to give to the secrets we have made above ??

->

- **SSH_HOST** -> This is simply the url to go to your machine
- **SSH_USERNAME** -> This the simply the os which your machine has (ex -> ubuntu, etc..)
- **SSH_KEY** -> Give the key (basically this is the password for entering to your machine)
- **Point userapp.your_domain.com to the IP of the server**

Now here the problem is that see `// 2` code using the above block of code will run the project on your machine but on the port 3005 (which is bad from the user point as user have to specify this in the url) so you will use **Nginx to deal with this and make them point via port 80(if http) or 443 (if https)**

- **Add Nginx reverse proxy to forward requests from userapp.your_domain.com to port on which the app is working**
 - **First install the Nginx** and then do the same thing as you were doing to setup Nginx in your project

and then the above part writing the below line of code

```
server {
    server_name userapp.100xdevs.com; // final things is to be seen at this url
    (basically add your custom domain name here)

    location / { // basically whatever is coming on the above DNS send them to the
    below localhost site
        proxy_pass http://localhost:3005;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;

        # Basic Authentication
        auth_basic "Restricted Content";
        auth_basic_user_file /etc/nginx/.htpasswd;
    }
}
```

```
listen 443 ssl; // managed by Certbot (use 80 if http and you have not learnt
certificate management)
ssl_certificate /etc/letsencrypt/live/userapp.100xdevs.com/fullchain.pem; //
managed by Certbot
ssl_certificate_key /etc/letsencrypt/live/userapp.100xdevs.com/privkey.pem; //
managed by Certbot
include /etc/letsencrypt/options-ssl-nginx.conf; // managed by Certbot
ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem; // managed by Certbot
}
```

As you have used port 443 (https) so

- **Install certbot and Refresh certificate**

```
sudo certbot --nginx
```

Assignments

1. Get a DB on [neon.tech](#)/ [RDS](#)/ [Aeveen](#) and add a DB migration step to the DB
2. Pass in the DB credentials while starting the docker image
3. Start the docker image so that it restarts if it goes down (similar to pm2)

Advanced -> The **Deploy to EC2 CI/CD part of the code** -> try replacing this title part code with something called as **Elastic Beanstalk** (present in the AWS) [An easy way to make application that can scale up and down by themselves]

Or try learning about **ASGs (Auto scaling groups)**